

SymPy Bug Report

1 Bug 16: Laplace Transform of $\text{Heaviside}(a - t)$ Uses the Wrong Cutoff

1.1 Minimal Reproducer

```
from sympy import Heaviside, exp, integrate, laplace_transform, symbols

t = symbols("t", real=True)
s = symbols("s", positive=True)
a = symbols("a")

F = laplace_transform(Heaviside(a - t), t, s, noconds=True)
print(F)
print(F.subs(a, 2))
print(integrate(exp(-s*t), (t, 0, 2)))
```

1.2 Observed Behavior

```
laplace_transform(Heaviside(a - t), t, s, noconds=False)
# (exp(-a*s)/s, 0, (a > 0) & (a < 0) & Ne(1/a, 0))

laplace_transform(Heaviside(a - t), t, s, noconds=True)
# exp(-a*s)/s

laplace_transform(Heaviside(a - t), t, s, noconds=True).subs(a, 2)
# exp(-2*s)/s

integrate(exp(-s*t), (t, 0, 2))
# 1/s - exp(-2*s)/s
```

The unconditional result exposed by `noconds=True` specializes to $\exp(-2s)/s$ when $a = 2$. This is not equal to the direct defining integral over the interval where $\text{Heaviside}(2 - t)$ is nonzero.

1.3 Expected Behavior

For a positive cutoff a , SymPy should specialize the transform to

$$\mathcal{L}\{\text{Heaviside}(a - t)\}(s) = \int_0^a e^{-st} dt = \frac{1 - \exp(-as)}{s}.$$

In particular, after substituting $a = 2$, the expected result is

$$\frac{1 - \exp(-2s)}{s},$$

equivalently $1/s - \exp(-2s)/s$. For symbolic a , the result should preserve the relevant assumptions or conditions rather than returning an unconditional expression that becomes false under positive numeric specialization.

1.4 Mathematical Explanation

For $a > 0$, the function $\text{Heaviside}(a - t)$ is a closing step: it is one for $0 \leq t < a$ and zero for $t > a$. The value at the single endpoint $t = a$ does not affect the Laplace integral. Therefore

$$\int_0^\infty e^{-st} \text{Heaviside}(a - t) dt = \int_0^a e^{-st} dt = \frac{1 - \exp(-as)}{s} \quad (a > 0, s > 0).$$

The returned expression $\exp(-as)/s$ is the transform of an opening step at $t = a$, not of the finite window $[0, a]$. The discrepancy is not a harmless rewriting: for $a = 2$, the difference between SymPy's result and the defining integral is

$$\frac{\exp(-2s)}{s} - \left(\frac{1 - \exp(-2s)}{s} \right) = \frac{2\exp(-2s) - 1}{s},$$

which is not identically zero.

1.5 Source-Level Diagnosis

The diagnosis identifies the relevant implementation in `sympy/integrals/laplace.py`. The public call `laplace_transform` constructs a `LaplaceTransform` object. Its `doit` method dispatches to `_laplace_transform`, which attempts simple table rules before the programmatic Heaviside-specific handling.

The simple-rule table contains two relevant patterns for `Heaviside(a*t-b)`, one for an opening step and one for a closing step:

```
(Heaviside(a*t-b), exp(-s*b/a)/s, And(a > 0, b > 0), ...)
(Heaviside(a*t-b), (1-exp(-s*b/a))/s, And(a < 0, b < 0), ...)
```

For `Heaviside(a-t)`, this pattern matches with coefficient -1 and $b = -a$, so the mathematically relevant positive-window rule is the second one. However, `_laplace_apply_simple_rules` applies a table entry only when its condition is exactly true. With symbolic a , neither condition is exactly true, so this simple-rule path fails. The later transform path returns the tuple

$$\left(\frac{\exp(-as)}{s}, 0, (a > 0) \wedge (a < 0) \wedge \text{Ne}(1/a, 0) \right).$$

The expression component corresponds to the wrong opening-step branch, and the condition component is contradictory. The diagnosis further notes that `LaplaceTransform.doit` drops the condition when `noconds=True`, so the invalid expression $\exp(-as)/s$ becomes the public result. A plausible fix direction, supported by the diagnosis, is to return a guarded conditional or `Piecewise` transform for symbolic a , or otherwise avoid returning a formula whose contradictory condition can be discarded by `noconds=True`. The existing programmatic Heaviside branch already appears to encode the positive-window logic when the sign of the cutoff is known.

1.6 Additional Failing Instances

The verification covers the representative cutoff $a = 2$ and five additional positive cutoffs. In each case, SymPy first computes the same symbolic transform $\exp(-as)/s$ and then substitutes a positive integer cutoff. The expected answer has the same form across the family because, for every positive cutoff c , $\text{Heaviside}(c - t)$ restricts the Laplace integral to $[0, c]$.

Input / Expression	SymPy behavior	Expected behavior
$\text{Heaviside}(a - t)$, then $a = 2$	$\exp(-2s)/s$	$(1 - \exp(-2s))/s$
$\text{Heaviside}(a - t)$, then $a = 1$	$\exp(-s)/s$	$(1 - \exp(-s))/s$
$\text{Heaviside}(a - t)$, then $a = 3$	$\exp(-3s)/s$	$(1 - \exp(-3s))/s$
$\text{Heaviside}(a - t)$, then $a = 4$	$\exp(-4s)/s$	$(1 - \exp(-4s))/s$
$\text{Heaviside}(a - t)$, then $a = 5$	$\exp(-5s)/s$	$(1 - \exp(-5s))/s$
$\text{Heaviside}(a - t)$, then $a = 6$	$\exp(-6s)/s$	$(1 - \exp(-6s))/s$

1.7 Related Issues Assessment

The deduplication artifact lists PR #25755 and issue #24561 as related public material and gives the verdict `family_known_specific_new`. Framed in the requested terms, this is a new instance of a known failure mode: the broader area of Laplace-transform handling for Heaviside conditions may have public precedent, but this exact symbolic closing-step case and its positive-cutoff specializations are previously unreported in the available evidence. The bug therefore remains individually actionable as a concrete issue and regression test.

1.8 Proposed SymPy Regression Test

```
import pytest

from sympy import Heaviside, exp, laplace_transform, simplify, symbols

@pytest.mark.parametrize("cutoff", [2, 1, 3, 4, 5, 6])
def test_laplace_transform_symbolic_heaviside_closing_step_after_substitution(cutoff):
    t = symbols("t", real=True)
    s = symbols("s", positive=True)
    a = symbols("a")

    symbolic_transform = laplace_transform(Heaviside(a - t), t, s, noconds=True)
    actual = symbolic_transform.subs(a, cutoff)
    expected = (1 - exp(-cutoff*s))/s

    assert simplify(actual - expected) == 0
```